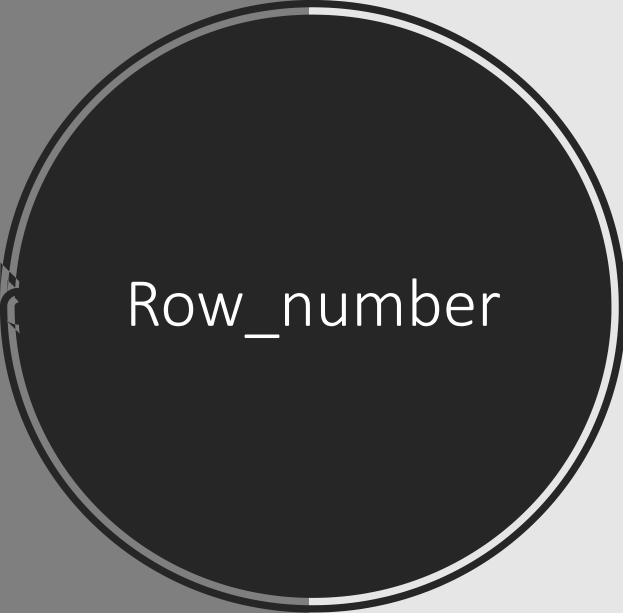


Ranking Function

Baza danych - MSSQL

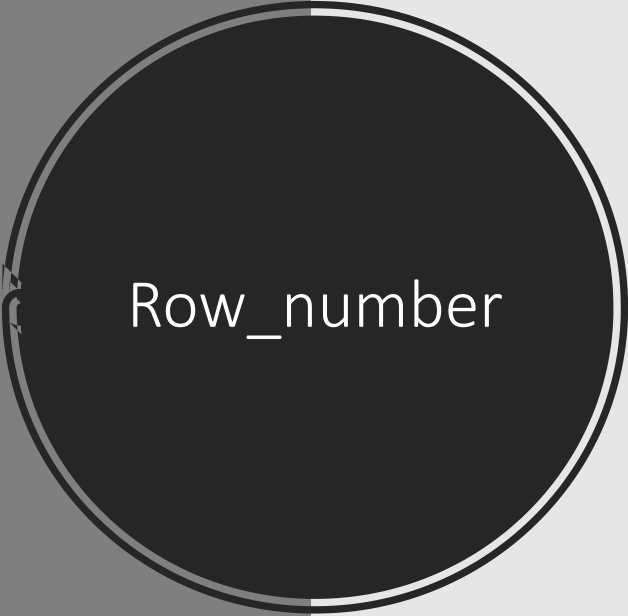
Przemysław Starosta - 2026



Row_number

Funkcja **row_number()** jest funkcją okienkową w SQL, która przypisuje unikalny, sekwencyjny numer każdemu wierszowi w zestawie wyników, zgodnie z określonym porządkiem sortowania. Jej główne zastosowania to:

- **Paginacja danych:** Umożliwia podział dużego zbioru wyników na mniejsze strony.
- **Ranking:** Służy do tworzenia rankingów lub porządkowania wierszy według określonych kryteriów.
- **Eliminacja duplikatów:** Może pomóc w identyfikacji i usunięciu zduplikowanych rekordów, poprzez wybranie tylko jednego wiersza dla danej grupy.



Row_number

	first_name	last_name	salary	row_number
1	TJ	Olson	2100.00	1
2	Steven	Markle	2200.00	2
3	Hazel	Philtanker	2200.00	3
4	Ki	Gee	2400.00	4
5	James	Landry	2400.00	5
6	Karen	Colmenares	2500.00	6
7	Joshua	Patel	2500.00	7
8	James	Marlow	2500.00	8

```
SELECT
    first_name,
    last_name,
    salary,
    row_number() over (order by salary) row_number
FROM
    employees
```



Partition

Operator **PARTITION BY** w funkcjach okienkowych w SQL służy do podziału zbioru wyników na oddzielne grupy, zwane partycjami. Dzięki temu funkcje okienkowe (takie jak ROW_NUMBER, RANK, SUM, AVG, itp.) wykonują obliczenia niezależnie dla każdej z tych partycji.

Kluczowe aspekty:

- **Izolacja obliczeń:** funkcja okienkowa oblicza swoje wyniki w obrębie każdej partycji osobno. Na przykład, używając PARTITION BY department, możesz obliczyć ranking pracowników w ramach każdego działu z osobna.
- **Bez zmiany liczby wierszy:** w przeciwieństwie do klauzuli GROUP BY, PARTITION BY nie redukuje liczby zwracanych wierszy. Każdy wiersz pozostaje w zbiorze wyników, ale funkcja okienkowa operuje tylko na wierszach należących do tej samej partycji.

Row_number - partition

	first_name	last_name	job_id	salary	row_number
1	William	Gietz	AC_ACCOUNT	8300.00	1
2	Shelley	Higgins	AC_MGR	12008.00	1
3	Jennifer	Whalen	AD_ASST	4400.00	1
4	Steven	King	AD_PRES	24000.00	1
5	Neena	Yang	AD_VP	17000.00	1
6	Lex	Garcia	AD_VP	17000.00	2
7	Luis	Popp	FI_ACCOUNT	6900.00	1
8	Ismael	Sciarra	FI_ACCOUNT	7700.00	2
9	Jose Manuel	Uman	FI_ACCOUNT	7800.00	3
10	John	Chen	FI_ACCOUNT	8200.00	4

```
SELECT
    first_name,
    last_name,
    job_id,
    salary,
    row_number()
        over (partition by job_id order by salary ) row_number
FROM
    employees
```



Rank

Funkcje **RANK()** oraz **DENSE_RANK()** to funkcje okienkowe w SQL, które służą do przydzielania pozycji (rang)

w zestawie wyników według określonego kryterium sortowania. Oto, jak działają i jakie mają zastosowania:

RANK()

- **Opis:**

Funkcja **RANK()** przypisuje rangę wierszom na podstawie podanego porządku.

W przypadku, gdy dwa lub więcej wierszy ma tę samą wartość sortującą, otrzymują one tę samą rangę.

- **Przerwy w rankingu:**

Po wystąpieniu remisów, kolejne rangi są pomijane. Na przykład, jeśli dwa rekordy uzyskają rangę 1, to kolejny rekord otrzyma rangę 3, a nie 2.

- **Przykładowe zastosowanie:**

Ranking pracowników według wynagrodzenia, gdzie przy równych wynagrodzeniach pracownicy mają tę samą pozycję, ale po nich następuje luka w kolejności.



Rank

	first_name	last_name	salary	rank
1	TJ	Olson	2100.00	1
2	Steven	Markle	2200.00	2
3	Hazel	Philtanker	2200.00	2
4	Ki	Gee	2400.00	4
5	James	Landry	2400.00	4
6	Karen	Colmenares	2500.00	6
7	Joshua	Patel	2500.00	6

```
SELECT
    first_name,
    last_name,
    salary,
    rank() over (order by salary) rank
FROM
    employees
```

Rank - partition

	first_name	last_name	job_id	salary	partition_rank
1	William	Gietz	AC_ACCOUNT	8300.00	1
2	Shelley	Higgins	AC_MGR	12008.00	1
3	Jennifer	Whalen	AD_ASST	4400.00	1
4	Steven	King	AD_PRES	24000.00	1
5	Neena	Yang	AD_VP	17000.00	1
6	Lex	Garcia	AD_VP	17000.00	1
7	Luis	Popp	FI_ACCOUNT	6900.00	1
8	Ismael	Sciarra	FI_ACCOUNT	7700.00	2
9	Jose Manuel	Uman	FI_ACCOUNT	7800.00	3
10	John	Chen	FI_ACCOUNT	8200.00	4

```
SELECT
    first_name,
    last_name,
    job_id,
    salary,
    rank() over (partition by job_id order by salary)
                partition_rank
FROM
    employees
```




Dense_rank

DENSE_RANK()

- **Opis:**

Funkcja **DENSE_RANK()** również przypisuje rangę wierszom, ale w odróżnieniu od **RANK()**, nie pomija kolejnych wartości.

- **Brak przerw:**

Przy remisach wszystkie rekordy otrzymują tę samą rangę, a kolejny unikalny rekord dostaje następną liczbę w sekwencji. Na przykład, jeśli dwa rekordy mają rangę 1, to następny rekord otrzyma rangę 2.

- **Przykładowe zastosowanie:**

Gdy potrzebujemy ciągłego rankingu bez luk, co jest przydatne w raportach lub przydzielaniu pozycji w rankingach.

Dense_rank

	first_name	last_name	salary	dense_rank
1	TJ	Olson	2100.00	1
2	Steven	Markle	2200.00	2
3	Hazel	Philtanker	2200.00	2
4	Ki	Gee	2400.00	3
5	James	Landry	2400.00	3
6	Karen	Colmenares	2500.00	4
7	Joshua	Patel	2500.00	4
8	James	Marlow	2500.00	4

```
SELECT
    first_name,
    last_name,
    salary,
    dense_rank() over (order by salary) dense_rank
FROM
    employees
```

Dense_rank - partition

	first_name	last_name	job_id	salary	partition_dense_rank
1	William	Gietz	AC_ACCOUNT	8300.00	1
2	Shelley	Higgins	AC_MGR	12008.00	1
3	Jennifer	Whalen	AD_ASST	4400.00	1
4	Steven	King	AD PRES	24000.00	1
5	Neena	Yang	AD_VP	17000.00	1
6	Lex	Garcia	AD_VP	17000.00	1
7	Luis	Popp	FI_ACCOUNT	6900.00	1
8	Ismael	Sciarra	FI_ACCOUNT	7700.00	2
9	Jose Manuel	Uman	FI_ACCOUNT	7800.00	3
10	John	Chen	FI_ACCOUNT	8200.00	4

```
SELECT
    first_name,
    last_name,
    job_id,
    salary,
    dense_rank() over (partition by job_id order by
                        salary) partition_dense_rank
FROM
    employees
```

Rank vs Dense_rank

	first_name	last_name	salary	rank	dense_rank
1	TJ	Olson	2100.00	1	1
2	Steven	Markle	2200.00	2	2
3	Hazel	Philtanker	2200.00	2	2
4	Ki	Gee	2400.00	4	3
5	James	Landry	2400.00	4	3
6	Karen	Colmenares	2500.00	6	4
7	Joshua	Patel	2500.00	6	4
8	James	Marlow	2500.00	6	4
9	Peter	Vargas	2500.00	6	4

SELECT

first_name,

last_name,

salary,

rank() over (order by salary) rank,

dense_rank() over (order by salary) dense_rank

FROM

employees

Comparison - partition

	first_name	last_name	job_id	salary	partition_rank	partition_dense_rank
1	William	Gietz	AC_ACCOUNT	8300.00	1	1
2	Shelley	Higgins	AC_MGR	12008.00	1	1
3	Jennifer	Whalen	AD_ASST	4400.00	1	1
4	Steven	King	AD_PRES	24000.00	1	1
5	Neena	Yang	AD_VP	17000.00	1	1
6	Lex	Garcia	AD_VP	17000.00	1	1
7	Luis	Popp	FI_ACCOUNT	6900.00	1	1
8	Ismael	Sciarra	FI_ACCOUNT	7700.00	2	2
9	Jose Manuel	Uman	FI_ACCOUNT	7800.00	3	3
10	John	Chen	FI_ACCOUNT	8200.00	4	4
11	Daniel	Faviet	FI_ACCOUNT	9000.00	5	5

```
SELECT
    first_name,
    last_name,
    job_id,
    salary,
    rank() over (partition by job_id order by salary) partition_rank,
    dense_rank() over (partition by job_id order by salary)
                        partition_dense_rank
FROM
    employees
```



Cume_dist (with current row)

Funkcja CUME_DIST() (cumulative distribution) jest funkcją okienkową w SQL, która zwraca skumulowany rozkład danego wiersza w obrębie określonego okna (partycji) w zbiorze danych. Mówiąc prościej, oblicza ona, jaki ułamek (procent) wszystkich wierszy w danej grupie (oknie) ma wartość mniejszą lub równą wartości bieżącego wiersza.

Jak działa CUME_DIST()

- **Porządkowanie wierszy** - w pierwszym kroku określamy, według jakich kolumn chcemy porządkować wiersze, np. ORDER BY Salary DESC.
- **Wyznaczenie rozkładu skumulowanego** - dla każdego wiersza obliczany jest odsetek wierszy, które mają wartość mniejszą lub równą danej wartości.
 - Jeśli wiersz ma najwyższą wartość, np. największe wynagrodzenie, to CUME_DIST() zwykle zwróci niewielką wartość (zależy, ile osób ma taką samą wartość).
 - Jeśli wiersz ma najniższą wartość, wynik będzie najbliższy 1 (czyli 100%).
- **Wartości zwracane** - funkcja zwraca wartości z zakresu (0, 1], co interpretujemy jako procent skumulowany (przedział od powyżej 0 do 1 włącznie).

Cume_dist
(with current
row)

	val	cume_dist
1	100	0,25
2	200	0,5
3	300	0,75
4	600	1

```
select
    t1. val,
    cume_dist() over (order by t1.val)
cume_dist
from(
    SELECT
        100 as val
    union
    SELECT
        200 as val
    union
    SELECT
        300 as val
    union
    SELECT
        600 as val ) t1
```



Percent_rank (without current row)

Funkcja **PERCENT_RANK()** jest funkcją okienkową w SQL, która zwraca względną pozycję (rangę) danego wiersza w danym zbiorze (lub partycji) w postaci wartości od **0** do **1**. Innymi słowy, określa, gdzie wiersz plasuje się procentowo wśród wszystkich wierszy w obrębie ustalonego porządku sortowania.

Jak działa **PERCENT_RANK()**

- **Określenie kolejności (ORDER BY)** -najpierw definiuje się porządek sortowania (np. według kolumny Salary).
- **Obliczanie rankingu** - PERCENT_RANK() korzysta z funkcji RANK() do wyznaczenia rangi wiersza, a następnie przekształca tę rangę w przedział 0,1 według wzoru:
 - Pierwszy wiersz (o rank = 1) otrzyma zawsze wartość 0,
 - Ostatni wiersz (o rank równej liczbie wierszy w partycji) otrzyma wartość 1 (o ile w partycji jest więcej niż jeden wiersz).
- **Zakres zwracanych wartości**
 - 0 – oznacza, że dany wiersz jest na samym początku w danej kolejności (najniższa wartość sortująca),
 - 1 – oznacza, że wiersz jest na końcu (najwyższa wartość sortująca) i jest zarazem ostatni w danej partycji.
 - Dla wierszy pomiędzy początkiem i końcem – wartości są rozłożone w przedziale (0, 1), proporcjonalnie do ich rangi.

Percent_rank
(without
current row)

	val	percent_rank
1	100	0
2	200	0,3333333333333333
3	300	0,6666666666666667
4	600	1

```
select
    t1. val,
    percent_rank() over (order by t1.val)
percent_rank
from(
    SELECT
        100 as val
    union
    SELECT
        200 as val
    union
    SELECT
        300 as val
    union
    SELECT
        600 as val ) t1
```



Percent_rank vs Cume_dist

Różnica między PERCENT_RANK() a CUME_DIST()

- **CUME_DIST()** wyznacza, jaki ułamek (procent) wierszy ma wartość **mniejszą lub równą** bieżącemu wierszowi.
- **PERCENT_RANK()** przelicza **RANK()** na skalę od 0 do 1, co oznacza pozycję bieżącego wiersza w uporządkowanym zbiorze w ujęciu procentowym (z uwzględnieniem możliwych “skoków” przy remisach).

Wybór między tymi funkcjami zależy od potrzeb analitycznych:

- **PERCENT_RANK()** dobrze pokazuje, w jakim “procentowym” miejscu znajduje się dany wiersz w zbiorze (pierwszy wiersz zawsze 0, ostatni zawsze 1).
- **CUME_DIST()** wskazuje, jaki odsetek wierszy osiąga **co najwyżej** daną wartość (pierwszy wiersz może otrzymać wartość większą niż 0, jeśli wiele wierszy ma tę samą wartość).



Funkcja **NTILE()** w SQL jest funkcją okienkową, która dzieli zbiór wyników na określoną liczbę przedziałów (tzw. „kubeków” lub „wiaderek”). Dzięki temu możemy podzielić wiersze w uporządkowanym zestawie danych na równe (lub prawie równe) grupy.

Jak działa NTILE?

- **Określenie liczby przedziałów (n)** - w wywołaniu funkcji podajemy liczbę n, czyli liczbę przedziałów, na jakie chcemy podzielić wyniki.

NTILE(n) OVER (ORDER BY ...)

- **Porządkowanie wierszy** - najpierw określamy, w jakiej kolejności sortujemy dane w oknie (np. ORDER BY Salary DESC).
- **Przydzielanie do przedziałów** -następnie wiersze są rozdzielane na n grup. Każdy wiersz dostaje numer grupy w przedziale 1 do n.
 - Jeśli zbiór dzieli się równo, to każda grupa ma tyle samo wierszy.
 - Jeśli nie dzieli się równo, to niektóre grupy mogą mieć o jeden wiersz więcej.
- **Wynik** - zwracana kolumna (z funkcji NTILE) zawiera numer grupy.



Ntile

Kiedy używać NTILE?

- **Analiza danych w segmentach:** np. dzielenie klientów według przychodu na określoną liczbę segmentów.
- **Tworzenie kwartyli/kwintyli/decyli:** np. dzielenie populacji na 4, 5 czy 10 równych grup w analizie statystycznej.
- **Porównywanie grup:** ułatwia badanie różnic pomiędzy poszczególnymi przedziałami (np. najlepszych, średnich i najgorszych).

Ntile

	first_name	last_name	job_id	salary	ntile
1	Diana	Nguyen	IT_PROG	4200.00	1
2	David	Williams	IT_PROG	4800.00	1
3	Valli	Jackson	IT_PROG	4800.00	1
4	Bruce	Miller	IT_PROG	6000.00	2
5	Luis	Popp	FI_ACCOUNT	6900.00	2
6	Ismael	Sciarra	FI_ACCOUNT	7700.00	2
7	Jose Manuel	Uman	FI_ACCOUNT	7800.00	3
8	John	Chen	FI_ACCOUNT	8200.00	3
9	Daniel	Faviet	FI_ACCOUNT	9000.00	4
10	Alexander	James	IT_PROG	9000.00	4

```
SELECT
    first_name,
    last_name,
    job_id,
    salary,
    ntile(4) over (order by salary ) ntile
FROM
    employees
where job_id in ('FI_ACCOUNT', 'IT_PROG')
```

Ntile - partiion

	first_name	last_name	job_id	salary	ntile
1	Luis	Popp	FI_ACCOUNT	6900.00	1
2	Ismael	Sciarra	FI_ACCOUNT	7700.00	1
3	Jose Manuel	Urman	FI_ACCOUNT	7800.00	2
4	John	Chen	FI_ACCOUNT	8200.00	3
5	Daniel	Faviet	FI_ACCOUNT	9000.00	4
6	Diana	Nguyen	IT_PROG	4200.00	1
7	David	Williams	IT_PROG	4800.00	1
8	Valli	Jackson	IT_PROG	4800.00	2
9	Bruce	Miller	IT_PROG	6000.00	3
10	Alexander	James	IT_PROG	9000.00	4

```
SELECT
    first_name,
    last_name,
    job_id,
    salary,
    ntile(4) over (partition by job_id order by salary ) ntile
FROM
    employees
where job_id in ('FI_ACCOUNT', 'IT_PROG')
```